A Comprehensive Performance Analysis of Apache Hadoop and Apache Spark for Large Scale Data Sets Using HiBench

Gökay Gülsoy 323078045

CENG544 Large-Scale Data Management

November 19, 2025

Big Data Analytics

- Big Data analytics for storing, processing, and analyzing large-scale datasets has become an essential tool for the industry. The advent of distributed computing frameworks such as Hadoop and Spark offers efficient solutions to analyze vast amounts of data, thanks to application programming interface (API) availability.

Research Question

- Both of these frameworks have more than 150 parameters, and the combination of these parameters has a huge impact on cluster performance.

Research Question

- Both of these frameworks have more than 150 parameters, and the combination of these parameters has a huge impact on cluster performance.
- The default system parameters help system administrators deploy their system applications without much effort.

Research Question

- Both of these frameworks have more than 150 parameters, and the combination of these parameters has a huge impact on cluster performance.
- The default system parameters help system administrators deploy their system applications without much effort.
- However, an open question remains: can new parameter selection improve cluster performance on large datasets?

Purpose of Study

- This study investigates the most impacting parameters, under resource utilization, input splits, and shuffle to compare performance between Hadoop and Spark using a cluster implemented in a laboratory.

Evaluation Strategy

- In order to evaluate the frameworks with comparative analysis, two workloads are selected:

Evaluation Strategy

- In order to evaluate the frameworks with comparative analysis, two workloads are selected:
- WordCount and TeraSort

Evaluation Strategy

- In order to evaluate the frameworks with comparative analysis, two workloads are selected:
- WordCount and TeraSort
- The performance metrics are carried out based on three criteria: execution time, throughput, and speedup.

Brief Look at Experimental Findings

- Both systems' performances heavily depend on the input data size and correct parameter selection.

Brief Look at Experimental Findings

- Both systems' performances heavily depend on the input data size and correct parameter selection.
- The analysis of results shows that Spark has better performance as compared to Hadoop when data sets are small.

Brief Look at Experimental Findings

- Both systems' performances heavily depend on the input data size and correct parameter selection.
- The analysis of results shows that Spark has better performance as compared to Hadoop when data sets are small.
- Spark achieved 2 times speedup in WordCount workloads and up to 14 times in TeraSort over Hadoop when default parameters are reconfigured.

Hadoop Components

- The large files are divided into several small blocks of equal size and distributed across the cluster for storage.

Hadoop Components

- The large files are divided into several small blocks of equal size and distributed across the cluster for storage.
- MapReduce and Hadoop distributed file system (HDFS) are core parts of the Hadoop system.

Hadoop Components

- The large files are divided into several small blocks of equal size and distributed across the cluster for storage.
- MapReduce and Hadoop distributed file system (HDFS) are core parts of the Hadoop system.
- Computing and storage work together across all nodes that compose a cluster of computers.

HDFS Working

- HDFS splits the files into small blocks and saves them on different nodes.

HDFS Working

- HDFS splits the files into small blocks and saves them on different nodes.
- There are two kinds of nodes on HDFS: data nodes (worker) and name-nodes (master nodes).

HDFS Working

- HDFS splits the files into small blocks and saves them on different nodes.
- There are two kinds of nodes on HDFS: data nodes (worker) and name-nodes (master nodes).
- All the operations, including delete, read, write, are based on these two types of nodes.

HDFS Working Cont'd

- Firstly, the name-node asks for access permission. If accepted, it will turn the file name into a list of HDFS block IDs, including data-nodes that saved the blocks related to that file.

HDFS Working Cont'd

- Firstly, the name-node asks for access permission. If accepted, it will turn the file name into a list of HDFS block IDs, including data-nodes that saved the blocks related to that file.
- The ID list will then be sent back to the client, and the users can do further operations based on that.

MapReduce Framework

- MapReduce is a framework used for distributed computing, parallel processing, and is designed purposely to write, read, and process bulky amounts of data.

MapReduce Framework

- MapReduce is a framework used for distributed computing, parallel processing, and is designed purposely to write, read, and process bulky amounts of data.
- This data processing framework is comprised of three stages: Map phase, Shuffle phase, and Reduce phase.

MapReduce Working

- MapReduce includes two fundamental operations: Mappers and Reducers:

MapReduce Working

- MapReduce includes two fundamental operations: Mappers and Reducers:
- Mappers will process files based on the map function and transform them into new key-value pairs. Then, new key-value pairs are assigned to different partitions based on their sorted keys.

MapReduce Working

- MapReduce includes two fundamental operations: Mappers and Reducers:
- Mappers will process files based on the map function and transform them into new key-value pairs. Then, new key-value pairs are assigned to different partitions based on their sorted keys.
- Combiner is an optional component, and its task can be recognized as a local reduce operation, which allows counting the values with the same key in advance to reduce the I/O pressure.

MapReduce Working Cont'd

- Finally, partitions will divide the intermediate key-value pairs into different pieces and transfer them to a reducer.

MapReduce Working Cont'd

- Finally, partitions will divide the intermediate key-value pairs into different pieces and transfer them to a reducer.
- Then the shuffle operation is executed, which transfers the mapper output data to the proper reducer.

MapReduce Working Cont'd

- Finally, partitions will divide the intermediate key-value pairs into different pieces and transfer them to a reducer.
- Then the shuffle operation is executed, which transfers the mapper output data to the proper reducer.
- Reducer then merges the output data into different final files, after that, these files are given to the reducer as input data and the reducer processes the data based on the reduce function and writes output back to the HDFS.

Hadoop MapReduce Architecture

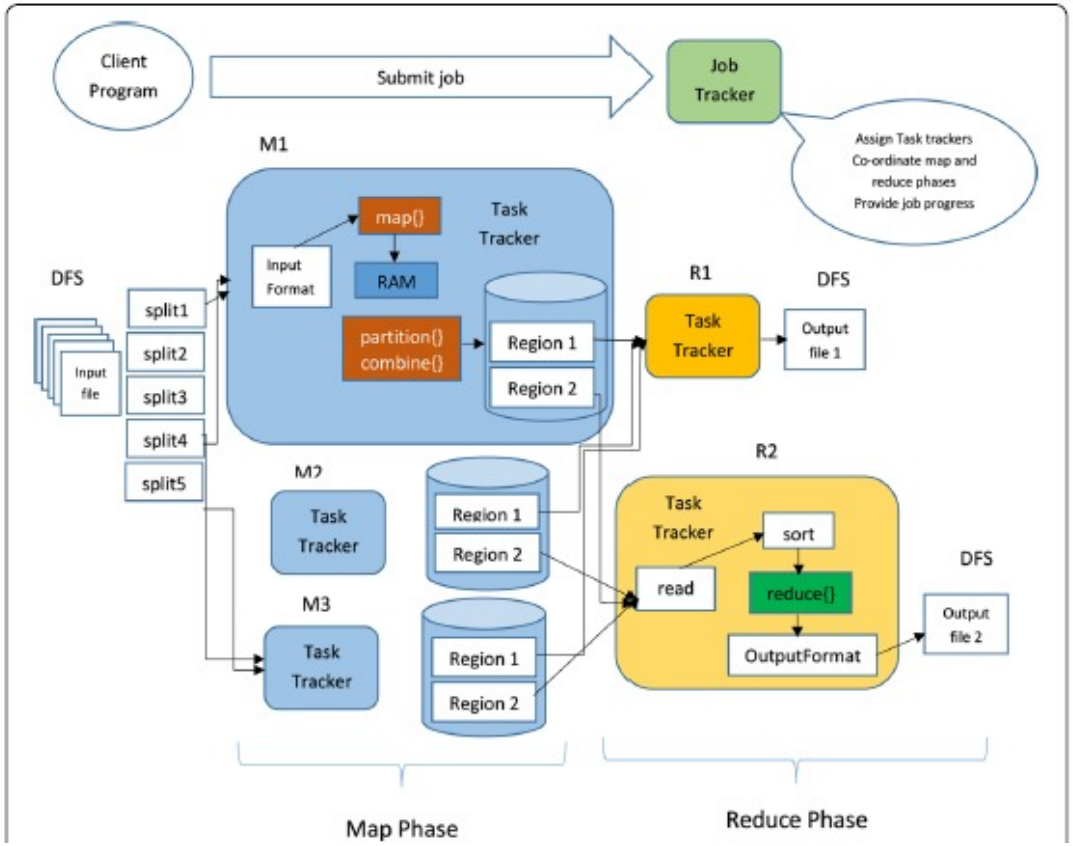


Figure: Hadoop MapReduce Architecture

Spark Working

- Spark proposed two important terms: Resilient Distributed Datasets (RDD) and Directed Acyclic Graph (DAG).
- When Spark runs on the Hadoop cluster, RDDs will be created on the HDFS in many formats supported by Hadoop.

Spark Working

- Spark proposed two important terms: Resilient Distributed Datasets (RDD) and Directed Acyclic Graph (DAG).
- When Spark runs on the Hadoop cluster, RDDs will be created on the HDFS in many formats supported by Hadoop.
- The DAG scheduler system expresses the dependencies of RDDs. Then each Spark job will create a DAG, and the scheduler will drive the graph into the different stages of tasks, and then tasks will be launched to the cluster.

Spark Working

- Spark proposed two important terms: Resilient Distributed Datasets (RDD) and Directed Acyclic Graph (DAG).
- When Spark runs on the Hadoop cluster, RDDs will be created on the HDFS in many formats supported by Hadoop.
- The DAG scheduler system expresses the dependencies of RDDs. Then each Spark job will create a DAG, and the scheduler will drive the graph into the different stages of tasks, and then tasks will be launched to the cluster.
- The DAG will be created in both map and reduce stages to fully express dependencies.

Spark Workflow

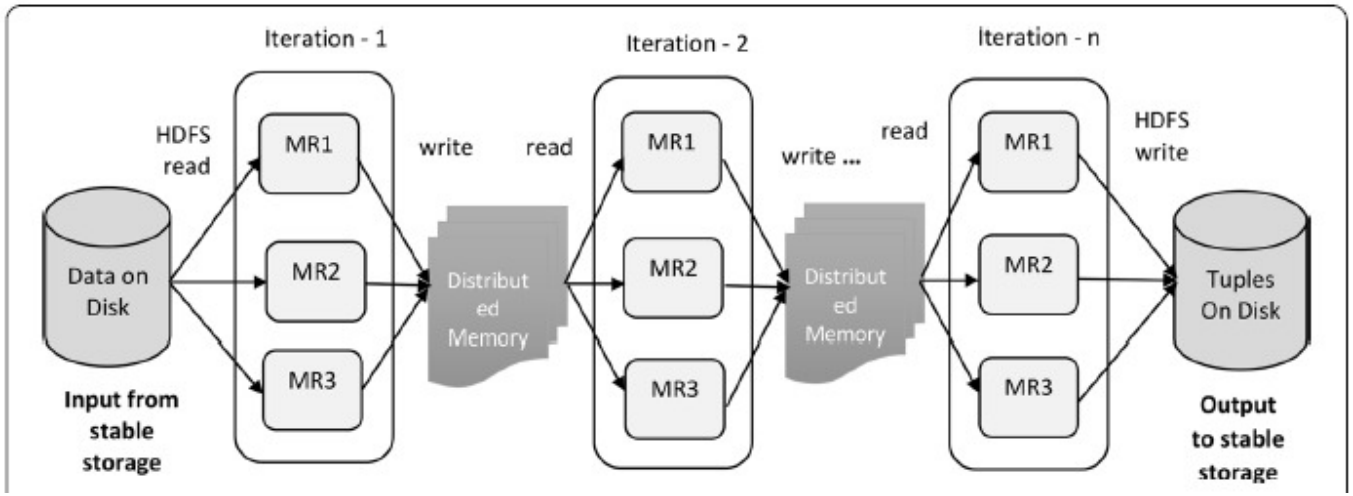


Figure: Spark Workflow

Experimental Setup

- Cluster performances measured based on MapReduce and Spark using the HiBench suite[1] .

[1] https://github.com/Intel-bigdata/HiBench

Experimental Setup

- Cluster performances measured based on MapReduce and Spark using the HiBench suite[1] .
- HiBench is a big data benchmark suite that helps evaluate different big data frameworks in terms of speed, throughput, and system resource utilization. It contains 29 workloads involving those for Hadoop and Spark.

[1] https://github.com/Intel-bigdata/HiBench

Experimental Setup

- Cluster performances measured based on MapReduce and Spark using the HiBench suite[1] .
- HiBench is a big data benchmark suite that helps evaluate different big data frameworks in terms of speed, throughput, and system resource utilization. It contains 29 workloads involving those for Hadoop and Spark.
- This study used two workloads, namely **WordCount (aggregation job)** and **TeraSort (shuffle job)** with large dataset size of 600 GB.

[1] https://github.com/Intel-bigdata/HiBench

Experimental Setup

- Cluster performances measured based on MapReduce and Spark using the HiBench suite[1] .
- HiBench is a big data benchmark suite that helps evaluate different big data frameworks in terms of speed, throughput, and system resource utilization. It contains 29 workloads involving those for Hadoop and Spark.
- This study used two workloads, namely **WordCount (aggregation job)** and **TeraSort (shuffle job)** with large dataset size of 600 GB.
- 600 GB input data was generated synthetically by the *TeraGen* function implemented in Java.

[1] https://github.com/Intel-bigdata/HiBench

Experimental Hadoop Cluster

Server configuration	Processor	2.9 GHz
	Main Memory	64 GB
	Local Storage	10 TB
Node configuration	CPU	Intel(R) Xeon(R) CPU E3-1231 v3 @ 3.40GHz
	Main Memory	32 GB
	Number of Nodes	10
	Local Storage	6 TB each, 60 TB total
	CPU cores	8 each, 80 total
Software	Operating System	Ubuntu 16.04.2 (GNU/Linux 4.13.0-37-generic×86 64)
	JDK	1.7.0
	Hadoop	2.4.0
	Spark	2.1.0
Workload	Micro Benchmarks	WordCount, and TeraSort

Figure: Experimental Hadoop Cluster Used

Hadoop and Spark Configurations

Configuration parameters category	Hadoop	Tuned values
Resource utilization	mapreduce.reduce.memory	8 GB
	mapred.reduce.task	16,384 MB, 25,600 MB
	mapreduce.reduce.cpu.vcores	4
Input split	mapred.min.split.size, mapred.max.split.size	128 MB (default), 256 MB, 512 MB, 1024 MB
Shuffle	i/o.sort.mb	25, 50, 75, 100
	i/o.sort.factor	512, 1024, 1536, 2047
	mapreduce.reduce.shuffle.parallelcopies	50, 100, 150, 200
	mapreduce.task.io.sort.factor	15, 30, 45, 60

Figure: Hadoop Configuration Parameters

Configuration parameters category	Spark	Tuned values
Resource utilization	num-executors	50
	executor-cores	4
	executor-memory	8 GB
Input split	spark.hadoop.MapReduce.input .fileinputformat.split.minsize	128 MB (default), 256MB, 512MB, 1024MB
Shuffle	spark.shuffle.file.buffer	16 k, 32 k (default), 48 k, 64 k
	spark.reducer.maxSizeInFlight	32 M, 48 M (default), 64 M, 96 M
	spark.hadoop.dfs.replication	1
	spark.default.parallelism	80, 100, 200, 300

Figure: Spark Configuration Parameters

Workload Details (WordCount)

- **WordCount** workload counts the number of occurrences of separate words from text or a sequence file.

Workload Details (WordCount)

- **WordCount** workload counts the number of occurrences of separate words from text or a sequence file.
- The input data is produced by the *RandomTextWriter* class. It splits into each word by using the map function and generates intermediate data for the reduce function as a key-value pair.

Workload Details (WordCount)

- **WordCount** workload counts the number of occurrences of separate words from text or a sequence file.
- The input data is produced by the *RandomTextWriter* class. It splits into each word by using the map function and generates intermediate data for the reduce function as a key-value pair.
- Then intermediate results are added up, generating the final word count by the reduce function.

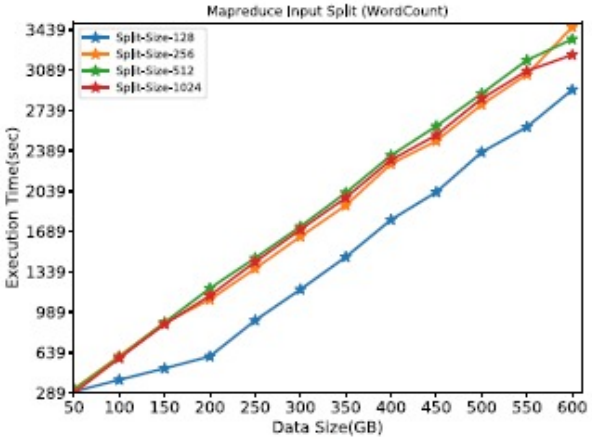
Workload Details (TeraSort)

- **TeraSort** package contains *TeraGen*, *TeraSort*, and *TeraValidate* functions.

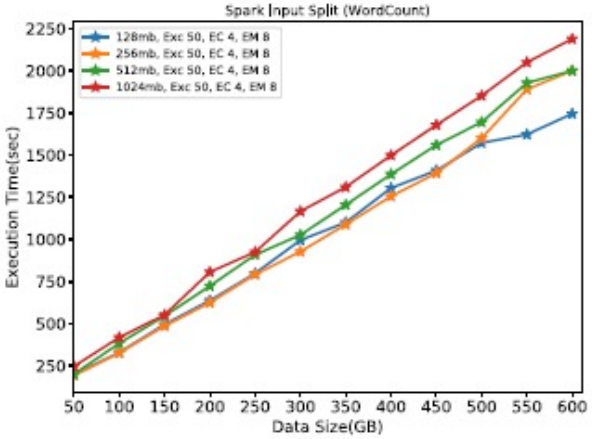
Workload Details (TeraSort)

- **TeraSort** package contains *TeraGen*, *TeraSort*, and *TeraValidate* functions.
- Its overall task is to sort the generated data using MapReduce and validate the output for sorted data.

Execution time Comparisons (WordCount Input Split)



a



c

Figure: The Performance of the Input Split Phase in WordCount

- Default input split size for MapReduce WordCount is 128 MB.
- Default parameter obtained better execution time for entire data sizes in MapReduce.
- Default input split size for Spark is 128 MB.
- 256 MB split size outperforms the default setup until 450 GB data sizes.

Execution time Comparisons (WordCount Shuffle)

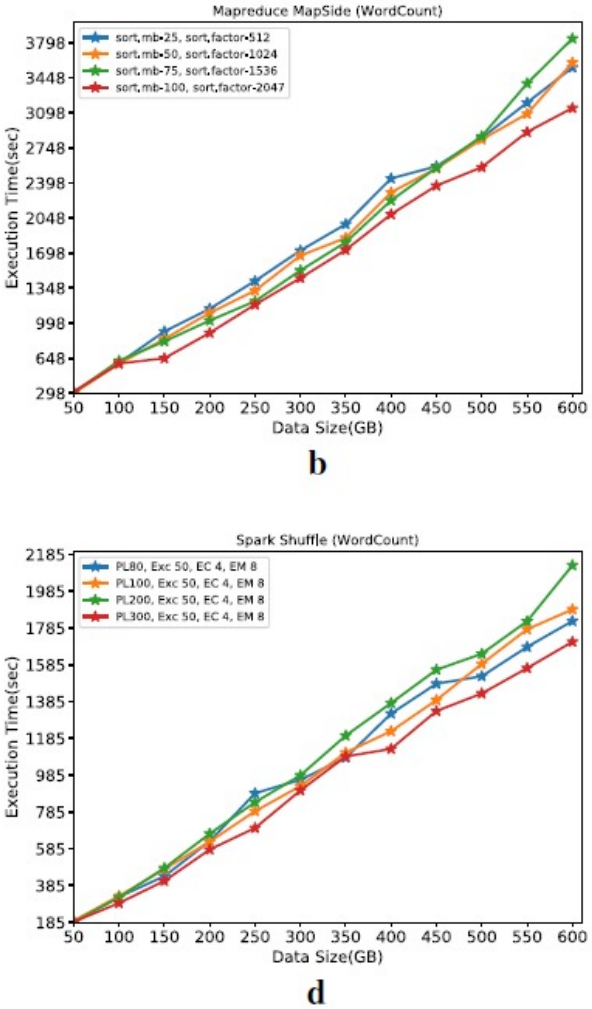


Figure: The Performance of the Shuffle Phase in WordCount

- Default shuffle parameter for MapReduce WordCount is (sort.mb 100, sort.factor 2047).
- Default parameter obtained better execution time for entire data sizes in MapReduce.
- Default shuffle parameter for Spark is PL100 object.
- when PL value is set to 300, best performance is achieved for sizes larger than 400 GB.

Execution time Comparisons (TeraSort InputSplit)

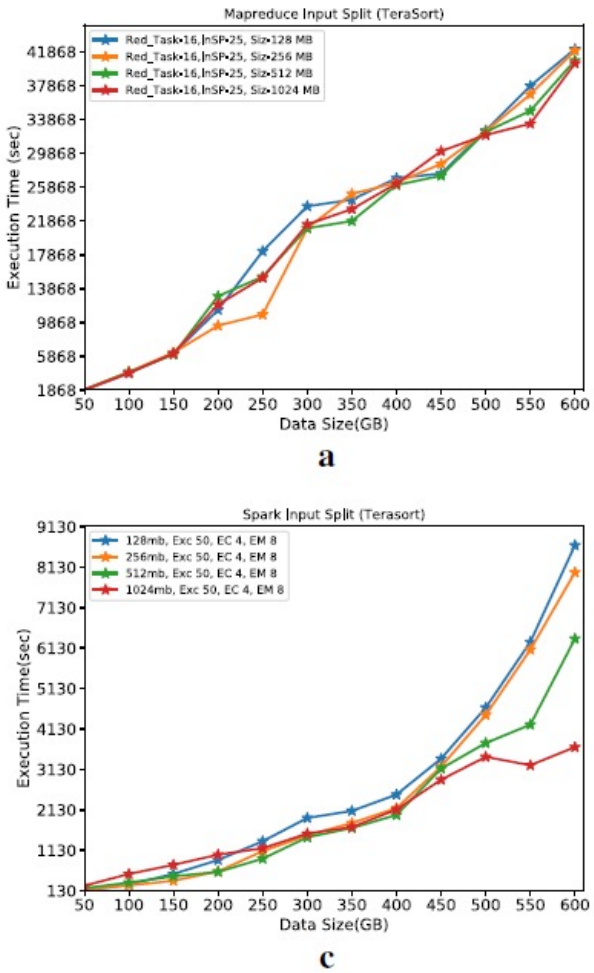
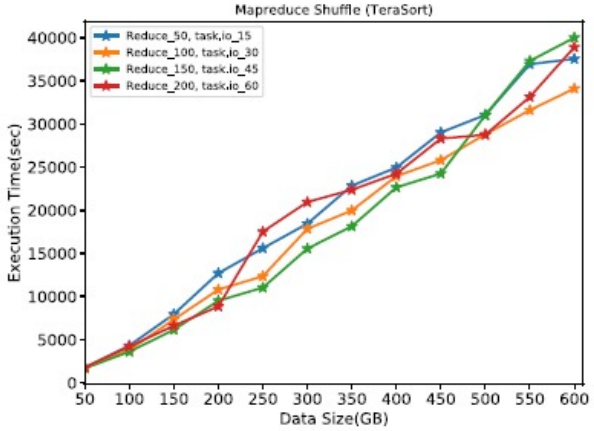


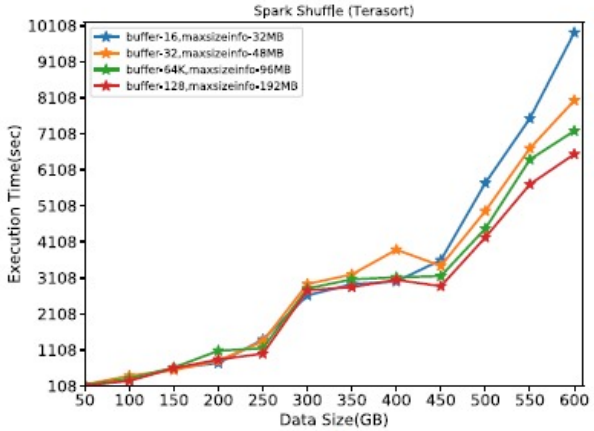
Figure: The Performance of the Input Split Phase in TeraSort

- Default input split parameter for MapReduce TeraSort is 128 MB.
- Split size of 256 MB performed better than default parameter datasets with size up to 300 GB.
- Default block size parameter for Spark is 128 MB.
- block size of 512 MB and 1024 MB performed better sizes up to 500 GB. Also increase of 4% by 1024 MB block size is observed for data size larger than 500 GB.

Execution time Comparisons (TeraSort Shuffle)



b



d

Figure: The Performance of the Shuffle Phase in TeraSort

- Default shuffle parameter for MapReduce TeraSort is (Reduce_100 and task.io_30).
- Configuration Reduce_150 and task.io_45) performed better than the default parameter for sizes up to 450 GB. After size of 450 GB, default configuration outperformed all others.
- Default shuffle parameter for Spark TeraSort is (buffer = 32, spark.reducer.maxSizeInflight = 48 MB).
- Configuration (buffer = 128, spark.reducer.maxSizeInflight = 192 MB) outperformed all other settings up to 600 GB data sizes.

Best Execution Times for WordCount Workload

	Split sizes (MB)	Execution time (s)
MapReduce input splits (WordCount)	128	2376
Spark input splits (WordCount)	256	1392
MapReduce shuffle (WordCount)	100	2371
Spark shuffle (WordCount)	300	1334

Figure: Best Execution Times of MapReduce and Spark with WordCount Workload

	Split sizes (MB)	Execution time (s)
MapReduce input splits (TeraSort)	256	21,014
Spark input splits (TeraSort)	512 & 1024	3780 & 3439
MapReduce shuffle (TeraSort)	150 & 45	24,250
Spark shuffle (TeraSort)	128 & 192	6540

Figure: Best Execution Times of MapReduce and Spark with TeraSort Workload

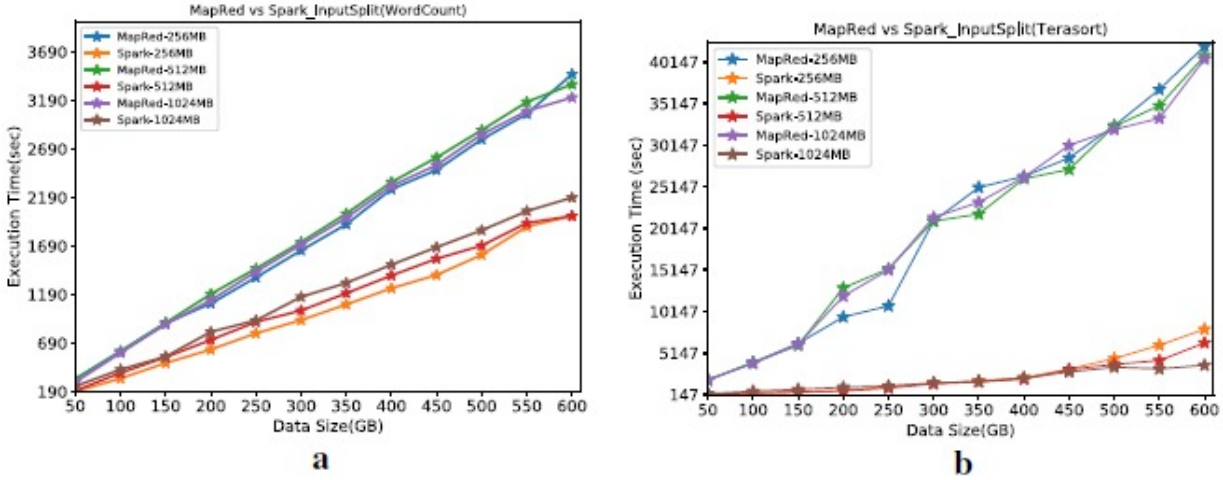


Figure: The Comparison of Hadoop and Spark with WordCount and TeraSort workload with varied input splits

- Spark with WordCount workloads showed higher execution performance by more than 2 times when data sizes are larger than 300 GB. For smaller data sizes performance gap is around 10 times.
- Unavoidable job action on the clusters could cause the dataset to be larger than the available RAM. MapReduce has slower data sharing capabilities and a longer time to read-write operations than Spark.

Throughput of WordCount and TeraSort Workload

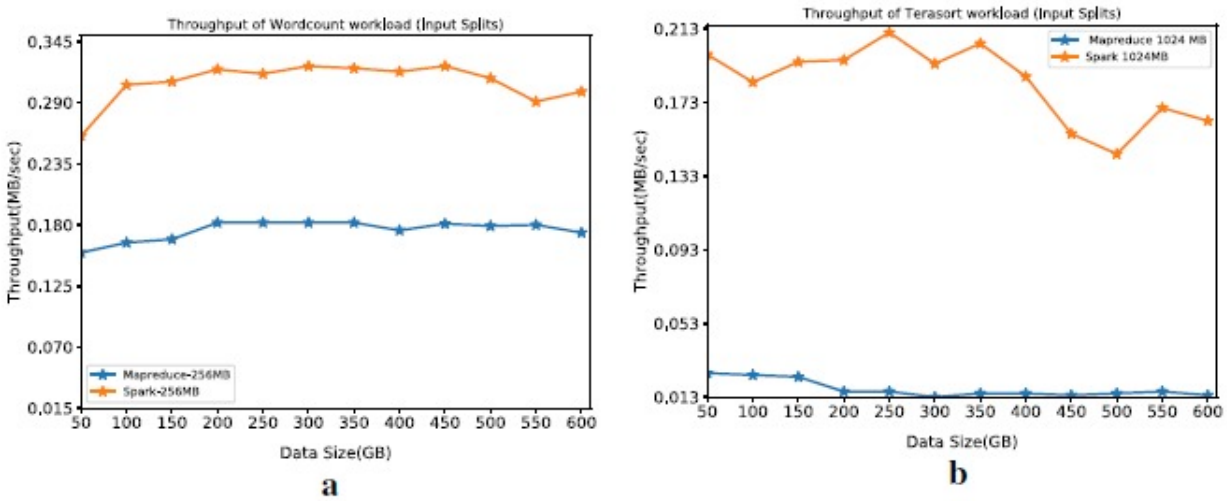


Figure: Throughput of WordCount and TeraSort workload

- MapReduce throughput performance for the TeraSort decreases slightly as the dataset size crosses beyond 200 GB.
- For the WordCount workload, the MapReduce throughput is almost linear.
- For the Spark TeraSort workload, it can be observed that the throughput is not constant. WordCount workload remains stable for most of the data sizes.

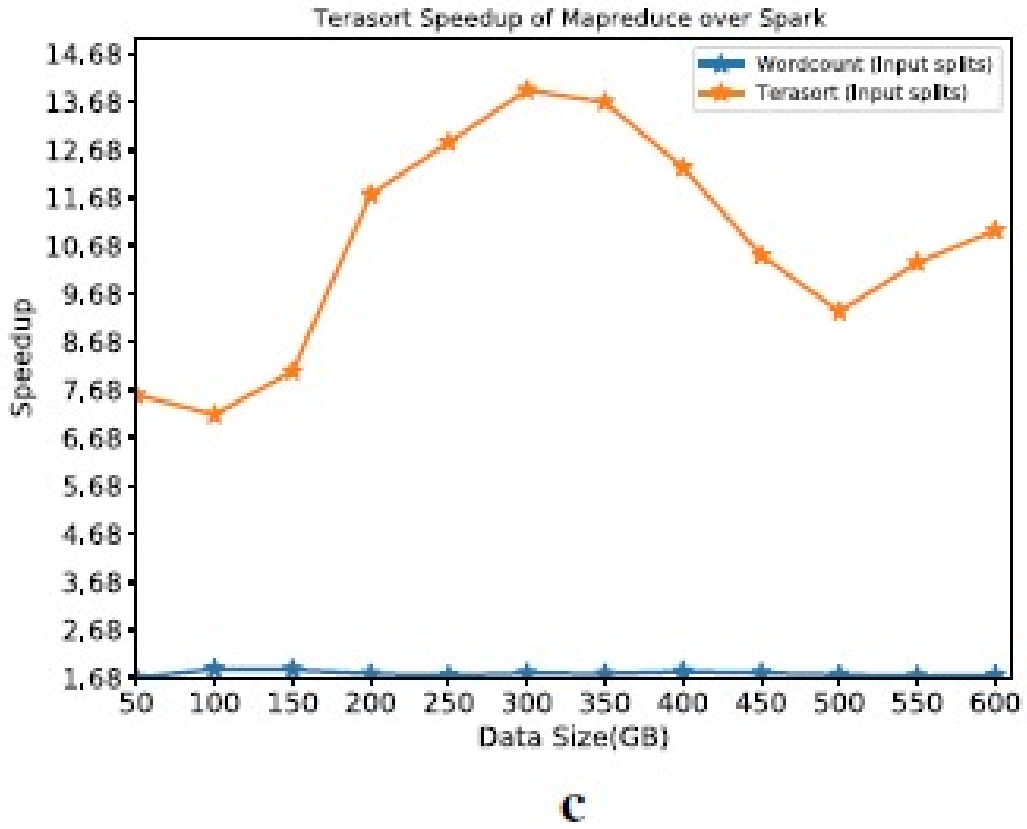


Figure: Spark over MapReduce speedup on input splits and shuffle

- TeraSort workload outperforms WordCount workload and achieved an all-time maximum of around 14 times.

Conclusions From Study

- Spark has better performance as compared to Hadoop by two times with WordCount workload and 14 times with TeraSort workloads, respectively, when default parameters are tuned with new values.
- Spark is more stable and faster than Hadoop because of Spark's data processing ability in memory instead of storing in disk for the map and reduce functions.
- Spark's performance degraded when the input data was larger.